Лабораторная работа №7

Архитектура ЭВМ

Альманасра Рами

Content

Список иллюстраций

1 Создание файла 4
2 Программа из листинга 5
3 запуск программы 5
4 изменение программы 6
5 запуск программы 6
6 изменение программы 7
7 запуск программы 7
8 Новая программа из листинга 8
9 Проверка работы программы из листинга 8
10 файл листинга 9
11 Удаление операнда из программы 10
12 ошибка в новом файле листинга 11
13 первое задание 12
14 провепка задания 14
15 второе задание 15
16 провепка второго задания 17

Список таблиц

Цель работы

Изучение команд условного и безусловного переходов. Приобретение навыков написания программ с использованием переходов. Знакомство с назначением и структурой файла листинга.

Задание

1. Реализация переходов в NASM
2. Изучение структуры файлов листинга
3. Самостоятельное написание программ на основе материалов лабораторной работы

Теоретическое введение

Для реализации ветвлений в ассемблере используются так называемые команды передачи управления или команды перехода. Можно выделить 2 типа переходов: • условный переход – выполнение или не выполнение перехода в определенную точку программы в зависимости от проверки условия. • безусловный переход – выполнение передачи управления в определенную точку программы без каких-либо условий.

Выполнение лабораторной работы

Реализация переходов в NASM

Создаю файл lab7-1 (Fig. -@fig:001).

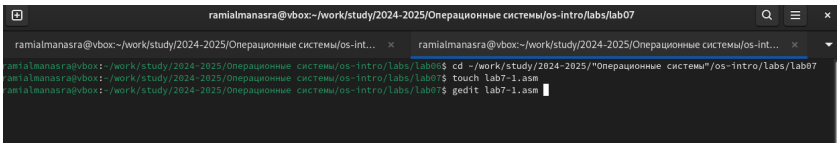


Рис. 1: Создание файла

Я копирую код из листинга в файл. (Fig. -@fig:002).

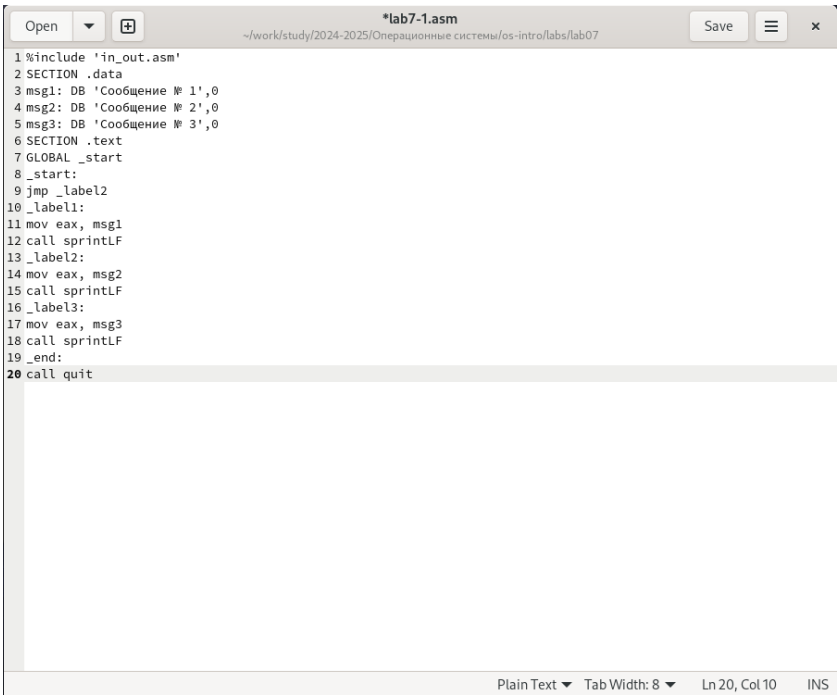


Рис. 2: Программа из листинга

При запуске программы я убедился, что безусловный переход действительно меняет порядок выполнения инструкций (Fig. -@fig:003).

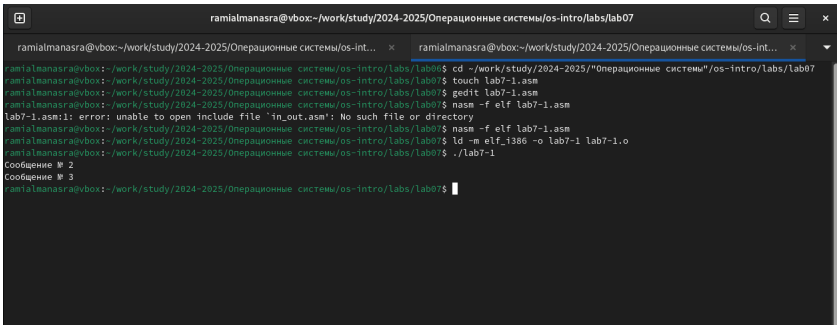


Рис. 3: запуск программы

Я меняю программу таким образом, что меняется порядок выполнения функций (Fig. -@fig:004).

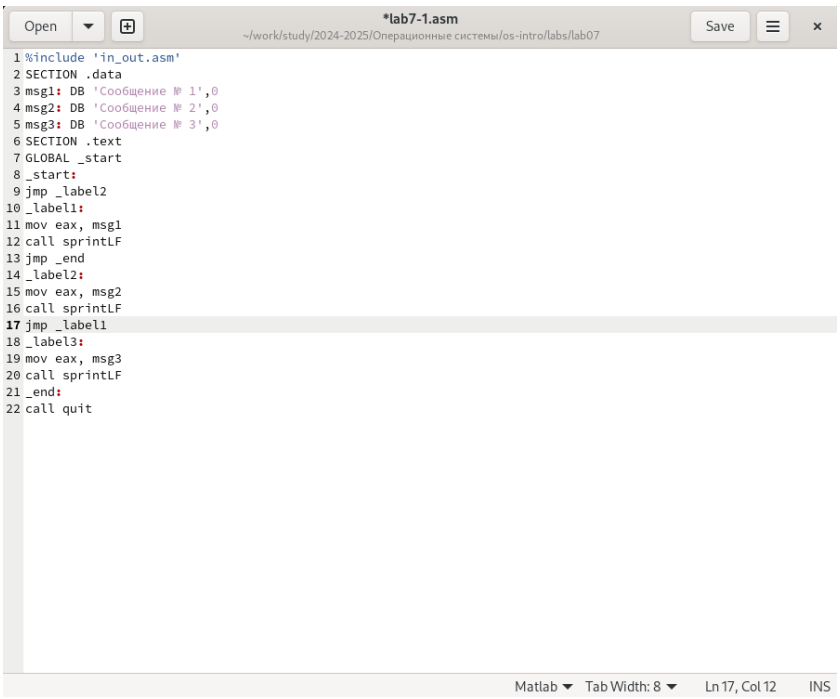


Рис. 4: изменение программы

запускаю программу и проверяю правильность внесенных изменений (Fig. -@fig:005).

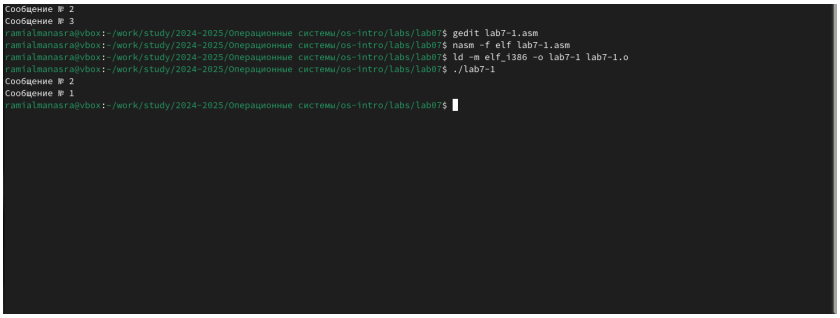


Рис. 5: запуск программы

Теперь изменяю текст программы так, чтобы все три сообщения отображались в обратном порядке (Fig. -@fig:006).

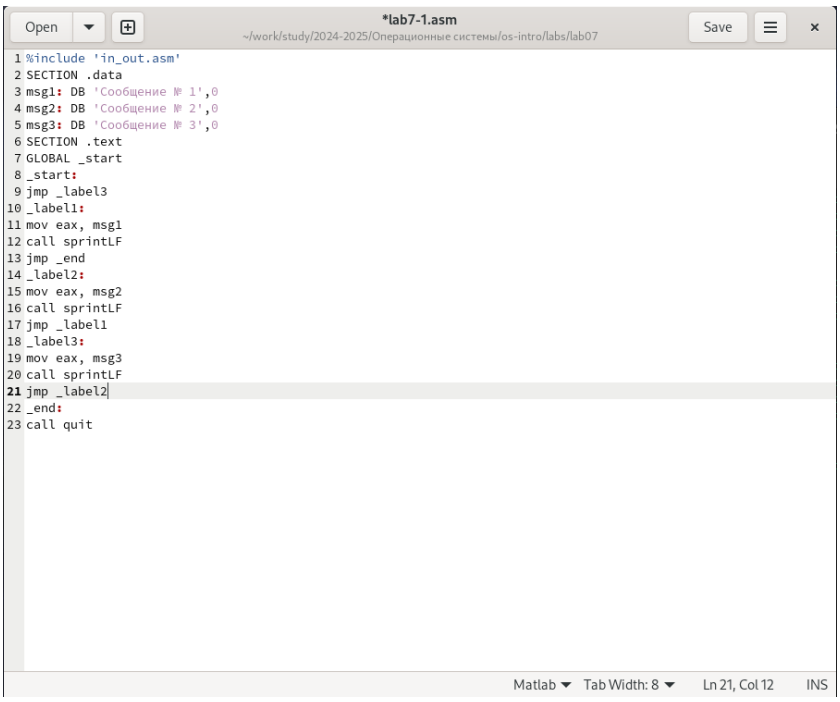


Рис. 6: изменение программы

Программа отображает сообщения в нужном мне порядке (Fig. -@fig:007).

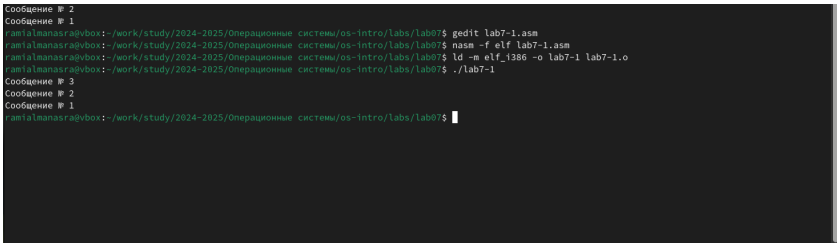


Рис. 7: запуск программы

Я создаю новый файл и вставляю в него код из следующего листинга (Fig. -@fig:008).

```
%include 'in_out.asm'
section .data
msg1 db 'Введите B: ',0h
msg2 db "Наибольшее число: ",0h
A dd '20'
C dd '50'
section .bss
max resb 10
B resb 10
section .text
global _start
_start:
mov eax,msg1
call sprint
mov ecx,B
mov edx,10
call sread
mov eax,B
call atoi
mov [B],eax
mov ecx,[A]
mov [max],ecx
cmp ecx,[C]
jg check_B
mov ecx,[C]
mov [max],ecx
check_B:
mov eax,max
call atoi
mov [max],eax
mov ecx,[max]
cmp ecx,[B]
jg fin
mov ecx,[B]
mov [max],ecx
fin:
mov eax, msg2
call sprint
mov eax,[max]
call iprintLF
call quit
```

Рис. 8: Новая программа из листинга

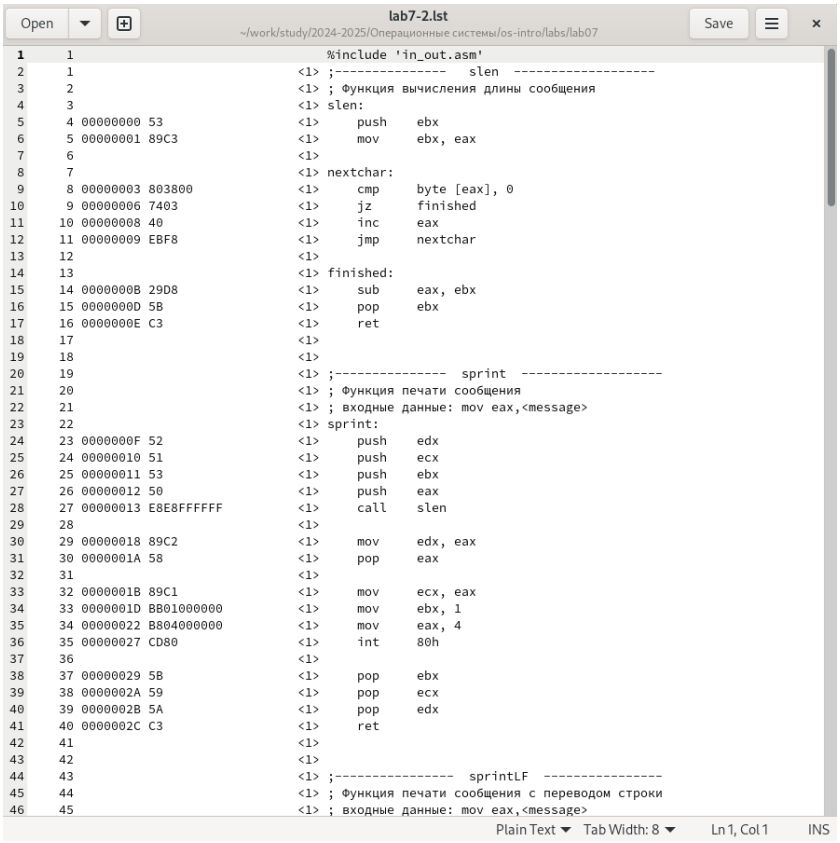
Программа выводит значение переменной с максимальным значением, проверяю работу программы с разными входными данными (Fig. -@fig:009).

```
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab07$ gedit lab7-2.asm
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab07$ nasm -f elf lab7-2.asm
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab07$ ld -m elf_i386 -o lab7-2 lab7-2.o
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab07$ ./lab7-2
Введите B: 5
Наибольшее число: 50
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab07$ 30
bash: 30: command not found...
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab07$ ./lab7-2
Введите B: 30
Наибольшее число: 50
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab07$ ./lab7-2
Введите B: 70
Наибольшее число: 70
ramialmanasra@vbox:~/work/study/2024-2025/Операционные системы/os-intro/labs/lab07$
```

Рис. 9: Проверка работы программы из листинга

#Изучение структуры файлы листинга

Создаю файл листинга для программы из файла lab7-2.asm указав ключ -l и открываю его с помощью gedit (Fig. -@fig:010).



```
lab7-2.lst
~/work/study/2024-2025/Операционные системы/os-intro/labs/lab07

  1                                   %include 'in_out.asm'
  1                                <1> ;---------------   slen  -------------------
  2                                <1> ; Функция вычисления длины сообщения
  3                                <1> slen:
  4 00000000 53                    <1>     push    ebx
  5 00000001 89C3                  <1>     mov     ebx, eax
  6                                <1>
  7                                <1> nextchar:
  8 00000003 803800                <1>     cmp     byte [eax], 0
  9 00000006 7403                  <1>     jz      finished
 10 00000008 40                    <1>     inc     eax
 11 00000009 EBF8                  <1>     jmp     nextchar
 12                                <1>
 13                                <1> finished:
 14 0000000B 29D8                  <1>     sub     eax, ebx
 15 0000000D 5B                    <1>     pop     ebx
 16 0000000E C3                    <1>     ret
 17                                <1>
 18                                <1>
 19                                <1> ;---------------  sprint  -------------------
 20                                <1> ; Функция печати сообщения
 21                                <1> ; входные данные: mov eax,<message>
 22                                <1> sprint:
 23 0000000F 52                    <1>     push    edx
 24 00000010 51                    <1>     push    ecx
 25 00000011 53                    <1>     push    ebx
 26 00000012 50                    <1>     push    eax
 27 00000013 E8E8FFFFFF            <1>     call    slen
 28                                <1>
 29 00000018 89C2                  <1>     mov     edx, eax
 30 0000001A 58                    <1>     pop     eax
 31                                <1>
 32 0000001B 89C1                  <1>     mov     ecx, eax
 33 0000001D BB01000000            <1>     mov     ebx, 1
 34 00000022 B804000000            <1>     mov     eax, 4
 35 00000027 CD80                  <1>     int     80h
 36                                <1>
 37 00000029 5B                    <1>     pop     ebx
 38 0000002A 59                    <1>     pop     ecx
 39 0000002B 5A                    <1>     pop     edx
 40 0000002C C3                    <1>     ret
 41                                <1>
 42                                <1>
 43                                <1> ;---------------  sprintLF  ---------------
 44                                <1> ; Функция печати сообщения с переводом строки
 45                                <1> ; входные данные: mov eax,<message>
Plain Text   Tab Width: 8   Ln 1, Col 1   INS
```

Рис. 10: файл листинга

Первое значение в файле-листинге - это номер строки, и он может не совпадать с номером строки исходного файла. Второе вхождение - это адрес, смещение машинного кода относительно начала текущего сегмента, далее идет непосредственно сам машинный код, а строка завершается исходным текстом программы с комментариями.

Я удаляю один операнд из случайной инструкции, чтобы проверить поведение файла листинга (Fig. -@fig:011).

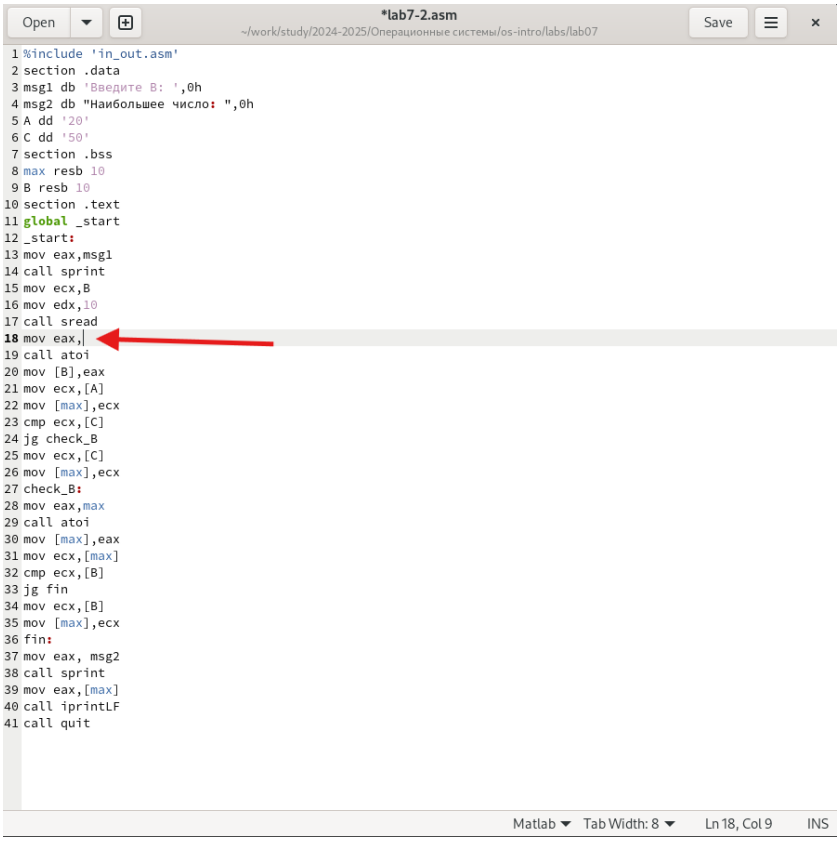
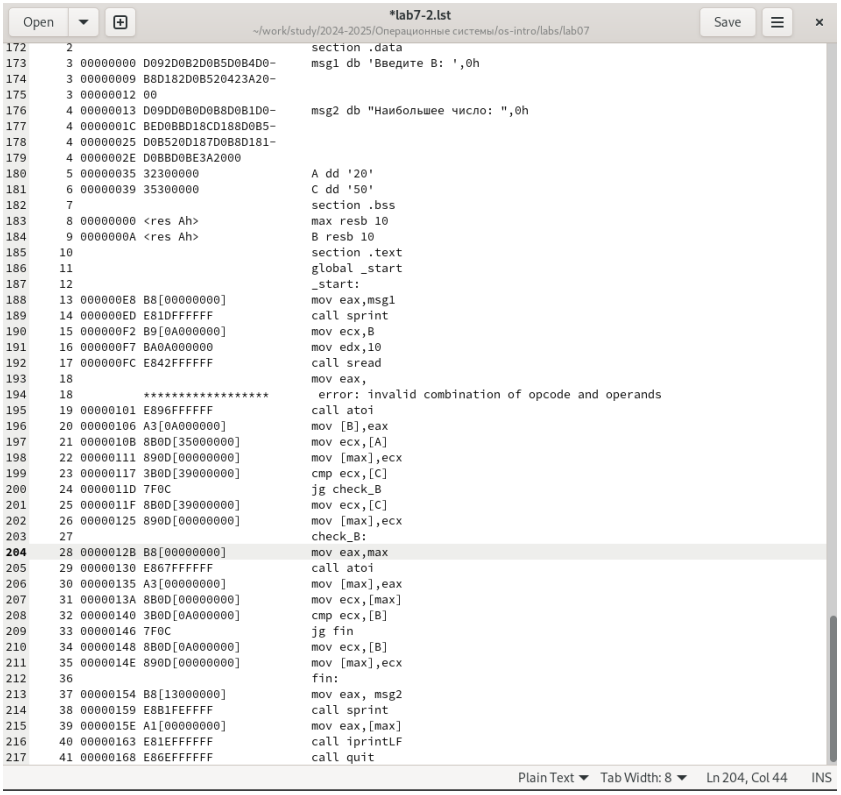


Рис. 11: Удаление операнда из программы

В новом файле листинга отображается ошибка error: invalid combination of opcode and operands. (Fig. -@fig:012).



```
Open   *lab7-2.lst   Save
~/work/study/2024-2025/Операционные системы/os-intro/labs/lab07

     2                                  section .data
     3 00000000 D092D0B2D0B5D0B4D0-     msg1 db 'Введите B: ',0h
     3 00000009 B8D182D0B520423A20-
     3 00000012 00
     4 00000013 D09DD0B0D0B8D0B1D0-     msg2 db "Наибольшее число: ",0h
     4 0000001C BED0BBD18CD188D0B5-
     4 00000025 D0B520D187D0B8D181-
     4 0000002E D0BBD0BE3A2000
     5 00000035 32300000                A dd '20'
     6 00000039 35300000                C dd '50'
     7                                  section .bss
     8 00000000 <res Ah>                max resb 10
     9 0000000A <res Ah>                B resb 10
    10                                  section .text
    11                                  global _start
    12                                  _start:
    13 000000E8 B8[00000000]            mov eax,msg1
    14 000000ED E81DFFFFFF              call sprint
    15 000000F2 B9[0A000000]            mov ecx,B
    16 000000F7 BA0A000000              mov edx,10
    17 000000FC E842FFFFFF              call sread
    18                                  mov eax,
    18          ******************       error: invalid combination of opcode and operands
    19 00000101 E896FFFFFF              call atoi
    20 00000106 A3[0A000000]            mov [B],eax
    21 0000010B 8B0D[35000000]          mov ecx,[A]
    22 00000111 890D[00000000]          mov [max],ecx
    23 00000117 3B0D[39000000]          cmp ecx,[C]
    24 0000011D 7F0C                    jg check_B
    25 0000011F 8B0D[39000000]          mov ecx,[C]
    26 00000125 890D[00000000]          mov [max],ecx
    27                                  check_B:
    28 0000012B B8[00000000]            mov eax,max
    29 00000130 E867FFFFFF              call atoi
    30 00000135 A3[00000000]            mov [max],eax
    31 0000013A 8B0D[00000000]          mov ecx,[max]
    32 00000140 3B0D[0A000000]          cmp ecx,[B]
    33 00000146 7F0C                    jg fin
    34 00000148 8B0D[0A000000]          mov ecx,[B]
    35 0000014E 890D[00000000]          mov [max],ecx
    36                                  fin:
    37 00000154 B8[13000000]            mov eax, msg2
    38 00000159 E8B1FEFFFF              call sprint
    39 0000015E A1[00000000]            mov eax,[max]
    40 00000163 E81EFFFFFF              call iprintLF
    41 00000168 E86EFFFFFF              call quit

Plain Text   Tab Width: 8   Ln 204, Col 44   INS
```

Рис. 12: ошибка в новом файле листинга

Задание для самостоятельной работы

Программа нахождения наименьшей из 3 целочисленных переменных:

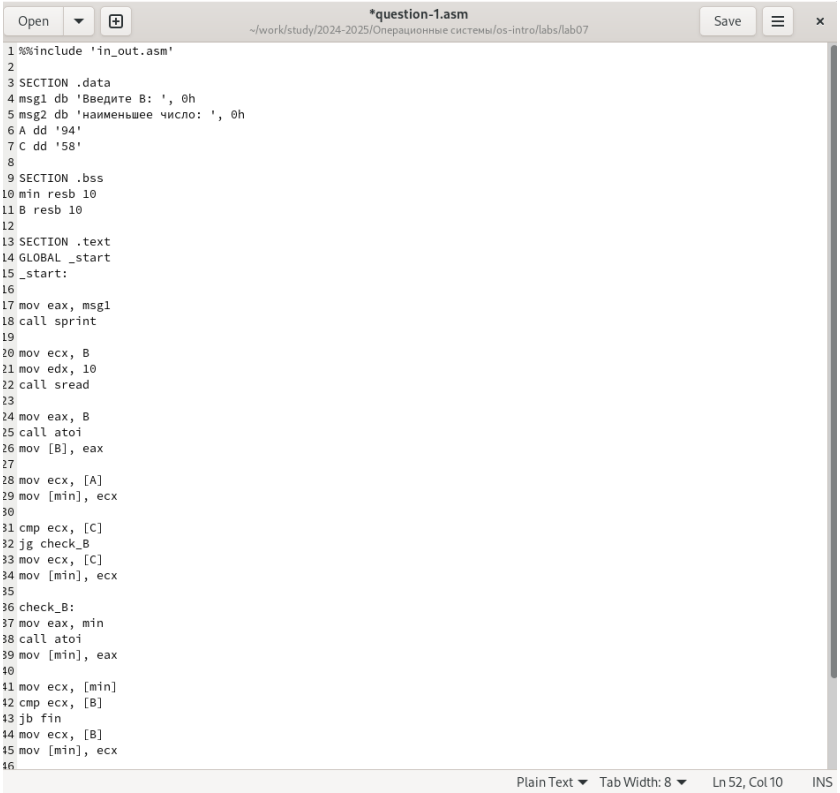


Рис. 13: первое задание

```
%%include 'in_out.asm'

SECTION .data
msg1 db 'Введите B: ', 0h
msg2 db 'наименьшее число: ', 0h
A dd '94'
C dd '58'

SECTION .bss
min resb 10
B resb 10

SECTION .text
```

```
GLOBAL _start
_start:

mov eax, msg1
call sprint

mov ecx, B
mov edx, 10
call sread

mov eax, B
call atoi
mov [B], eax

mov ecx, [A]
mov [min], ecx

cmp ecx, [C]
jg check_B
mov ecx, [C]
mov [min], ecx

check_B:
mov eax, min
call atoi
mov [min], eax

mov ecx, [min]
cmp ecx, [B]
```

```
jb fin
mov ecx, [B]
mov [min], ecx

fin:
mov eax, msg2
call sprint
mov eax, [min]
call iprintLF
call quit
```

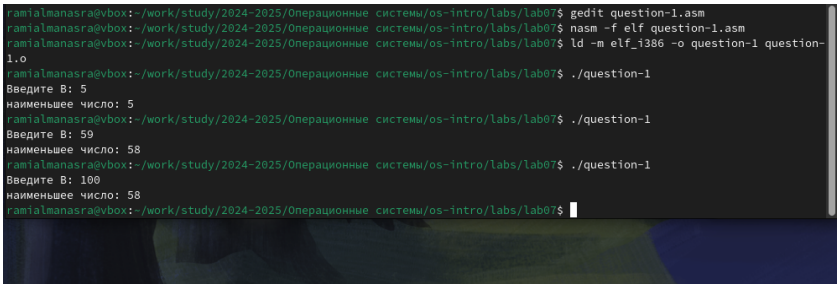


Рис. 14: провепка задания

пишу программу, которая будет вычислять значение заданной функции в соответствии с моим вариантом для переменных a и x, введенных с клавиатуры (Fig. -@fig:015).

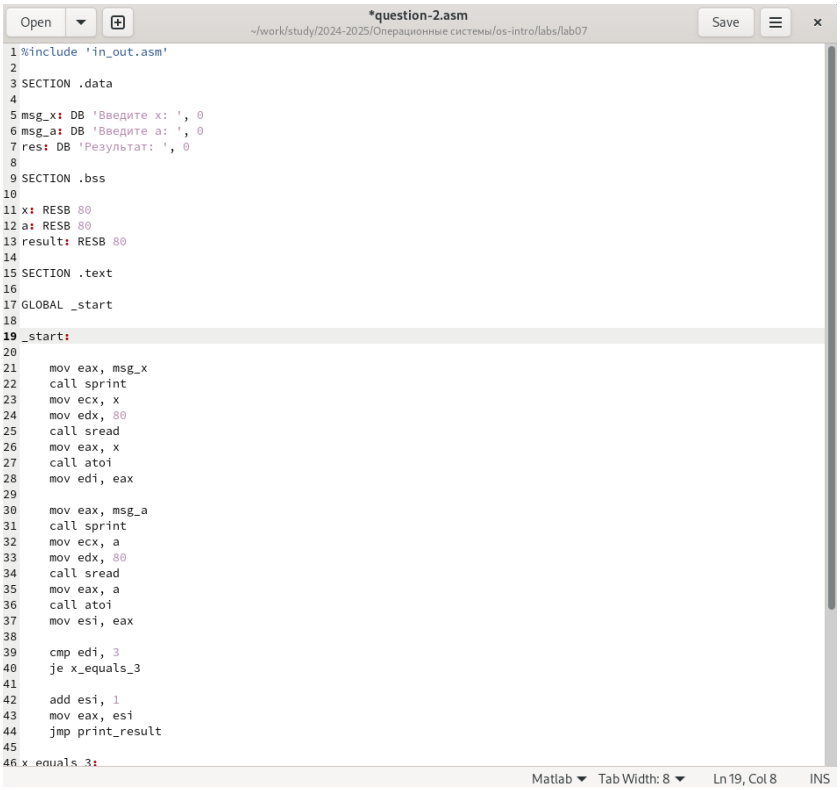


Рис. 15: второе задание

код задания:

```
%include 'in_out.asm'

SECTION .data

msg_x: DB 'Введите x: ', 0
msg_a: DB 'Введите a: ', 0
res: DB 'Результат: ', 0

SECTION .bss

x: RESB 80
a: RESB 80
```

```
result: RESB 80

SECTION .text

GLOBAL _start

_start:

    mov eax, msg_x
    call sprint
    mov ecx, x
    mov edx, 80
    call sread
    mov eax, x
    call atoi
    mov edi, eax

    mov eax, msg_a
    call sprint
    mov ecx, a
    mov edx, 80
    call sread
    mov eax, a
    call atoi
    mov esi, eax

    cmp edi, 3
    je x_equals_3
```

```
    add esi, 1
    mov eax, esi
    jmp print_result

x_equals_3:
    mov eax, edi
    mov ebx, 3
    imul eax, ebx

print_result:
    mov ebx, eax
    mov eax, res
    call sprint
    mov eax, ebx
    call iprintLF

    call quit
```

Проверяю прошрамму (Fig. -@fig:016).

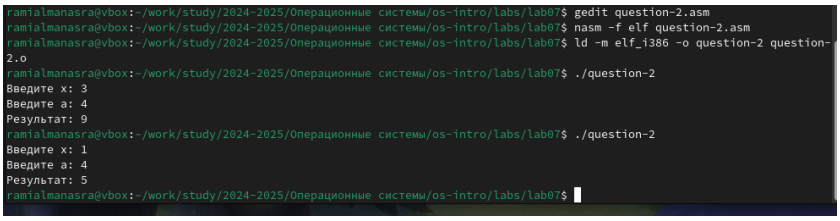


Рис. 16: провепка второго задания

Выводы

Во время лабораторной работы я изучил команды условных и безусловных переходов, а также приобрел навыки написания программ с использованием переходов, познакомился с назначением и структурой файлов листинга.

Список литературы

1. Course at RUDN University
2. Laboratory work No. 7
3. Programming in NASM assembler language by Stolyarov A. V.